

Smart Task Manager & Job Scheduler

Miyyapuram Sai Varun (BT2024146)

1 Problem Statement

The problem addressed in this project is to design a system capable of managing multiple user-submitted tasks efficiently while ensuring correctness, concurrency, and security. In real-world systems, tasks must be scheduled based on priority and timing, while handling multiple users simultaneously without data corruption.

This project aims to build a task scheduling system that demonstrates key Operating System concepts such as process creation, synchronization, inter-process communication, and client-server communication, while maintaining data consistency and role-based access control.

The goal of this project is to implement a task scheduling system that demonstrates core Operating System concepts such as process management, synchronization, inter-process communication, and client-server communication.

2 Project Description

The Smart Task Manager & Job Scheduler is a client-server based system where users can submit jobs such as backup, report generation, cleanup, and reminders. These jobs are executed asynchronously based on priority and delay.

The system is divided into four modules:

- Client: User interaction
- Server: Command handling and communication
- Scheduler: Job execution and management
- Storage: File persistence and logging

3 Features and Implementation

3.1 ADD_JOB

Command: ADD_JOB (type) (HIGH — MEDIUM — LOW) (delay)

Description: Allows users to submit a job.

Implementation:

- Validates input and role
- Creates job entry

- Uses fork() to execute job
- Stores in jobs.txt

Screenshot:

```
miyyapuram_saivarun@VICTUS1:/mnt/c/Users/varun/Downloads/project_OS$ ./client

SMART TASK MANAGER - Client

Type HELP after login for available commands.

[client] Connecting to 127.0.0.1:8080 ...

[client] Connected.

-----

After login, available commands:
ADD_JOB <type> <HIGH|MEDIUM|LOW> <delay>
  types: backup report cleanup reminder compile
VIEW_MY_JOBS
VIEW_ALL_JOBS
JOB_STATUS <id>
CANCEL_JOB <id>
CHANGE_PRIORITY <id> <HIGH|MEDIUM|LOW> [ADMIN]
VIEW_LOGS [ADMIN]
LOGOUT EXIT

-----

Welcome to Smart Task Manager
LOGIN <username> <password>

> LOGIN admin admin123
OK: Logged in as admin [ADMIN]

> ADD_JOB backup HIGH 200
OK: Job 1 submitted (backup HIGH delay=200s)
```

3.2 VIEW_MY_JOBS

Command: VIEW_MY_JOBS

Implementation:

- Filters jobs by user

Screenshot:

```
> VIEW_MY_JOBS
ID      OWNER      TYPE      PRIO      STATE      DELAY
---      -
1       admin      backup    HIGH      RUNNING    200
```

3.3 VIEW_ALL_JOBS

Command: VIEW_ALL_JOBS

Implementation:

- Displays all jobs

Screenshot:

```
> VIEW_ALL_JOBS
ID      OWNER      TYPE      PRIO      STATE      DELAY
---      -
1      admin      backup    HIGH     RUNNING    200
2      alice      compile   LOW      RUNNING    50
```

3.4 JOB_STATUS

Command: JOB_STATUS (ID)

Implementation:

- Retrieves job state

Screenshot:

```
> JOB_STATUS 1
Job 1 | owner=admin | type=backup | prio=HIGH | state=RUNNING | delay=200s
```

3.5 CANCEL_JOB

Command: CANCEL_JOB (ID)

Implementation:

- Cancels job if allowed

Screenshot:

```
> CANCEL_JOB 3
OK: Job 3 cancelled.

> VIEW_ALL_JOBS
ID      OWNER      TYPE      PRIO      STATE      DELAY
---      -
1      admin      backup    HIGH     RUNNING    200
2      alice      compile   LOW      RUNNING    50
3      admin      cleanup   LOW      CANCELLED   20
```

3.6 CHANGE_PRIORITY

Command: CHANGE_PRIORITY (ID) (HIGH — MEDIUM — LOW)

Implementation:

- Admin updates priority

Screenshot:

```
> ADD_JOB report MEDIUM 30
OK: Job 4 submitted (report MEDIUM delay=30s)

> CHANGE_PRIORITY 4 HIGH
OK: Job 4 priority changed to HIGH.

> VIEW_ALL_JOBS
ID    OWNER    TYPE    PRIO    STATE    DELAY
---    -
1     admin    backup  HIGH    RUNNING  200
2     alice    compile LOW     COMPLETED 50
3     admin    cleanup LOW     CANCELLED 20
4     admin    report  HIGH    RUNNING  30
```

3.7 VIEW_LOGS

Command: VIEW_LOGS

Implementation:

- Displays logs from file

Screenshot:

```
> VIEW_LOGS

[2026-04-30 22:52:35] USER=admin ACTION=LOGIN
[2026-04-30 22:53:01] USER=admin ACTION=backup
[2026-04-30 22:53:02] USER=admin ACTION=JOB 1 STARTED (pid=2051)
[2026-04-30 22:53:36] USER=alice ACTION=LOGIN
[2026-04-30 22:53:39] USER=alice ACTION=DISCONNECT
[2026-04-30 22:53:55] USER=alice ACTION=LOGIN
[2026-04-30 22:54:15] USER=alice ACTION=compile
[2026-04-30 22:54:16] USER=alice ACTION=JOB 2 STARTED (pid=2122)
[2026-04-30 22:54:45] USER=admin ACTION=cleanup
[2026-04-30 22:54:46] USER=admin ACTION=JOB 3 STARTED (pid=2127)
[2026-04-30 22:54:52] USER=admin ACTION=CANCEL JOB 3
[2026-04-30 22:55:09] USER=alice ACTION=JOB 2 COMPLETED
[2026-04-30 22:55:19] USER=admin ACTION=report
[2026-04-30 22:55:20] USER=admin ACTION=JOB 4 STARTED (pid=2132)
[2026-04-30 22:55:29] USER=admin ACTION=CHANGE_PRIORITY JOB 4 -> HIGH
```

3.8 LOGOUT and EXIT

Commands: LOGOUT, EXIT

Implementation:

- LOGOUT resets session
- EXIT closes connection

Screenshot:

```

Welcome to Smart Task Manager
LOGIN <username> <password>

> LOGIN admin admin123
OK: Logged in as admin [ADMIN]

> LOGOUT
Logged out.
LOGIN <username> <password>
> Welcome to Smart Task Manager
LOGIN <username> <password>

> [client] Logged out. You may log in again.
LOGIN alice alice123
OK: Logged in as alice [USER]

> EXIT
Bye!
[client] Disconnected.

```

4 Operating System Concepts Used

- **Role-Based Authorization:** Users are assigned roles (Admin, User, Guest), and access to commands is restricted based on permissions.
- **File Locking:** File locking is used while reading and writing to jobs.txt and logs.txt to prevent corruption during concurrent access.
- **Concurrency Control:** Multiple clients can connect simultaneously, and synchronization mechanisms such as mutexes ensure safe access to shared data.
- **Data Consistency:** Shared job data is protected to prevent race conditions and ensure correct updates of job states.
- **Socket Programming:** Communication between client and server is implemented using TCP sockets.
- **Inter-Process Communication (IPC):** Job execution is handled using fork(), and process completion is tracked using waitpid().

5 File Structure

- **client.c:** Handles user interaction
- **server.c:** Manages communication and command parsing

- **scheduler.c:** Executes jobs and manages scheduling
- **storage.c:** Handles file operations
- **common.h:** Shared definitions and structures

6 Challenges Faced and Solutions

- **Race Conditions:** Concurrent access to shared job data caused inconsistencies, which were resolved using mutex locks.
- **Zombie Processes:** Child processes were not being cleaned properly, which was fixed using `waitpid()`.
- **File Corruption:** Simultaneous file access led to issues, solved by implementing file locking.
- **Command Handling Errors:** Some commands were not functioning due to incorrect routing, which was fixed by properly mapping commands in `server.c`.
- **Function Duplication:** Duplicate implementations across files caused linker errors, which were resolved by restructuring the code into proper modules.

7 Conclusion

This project successfully demonstrates core Operating System concepts through a practical implementation. It integrates concurrency, process management, IPC, and synchronization into a real-world scheduling system.